

Finite State Machines and Sequential Circuit Synthesis

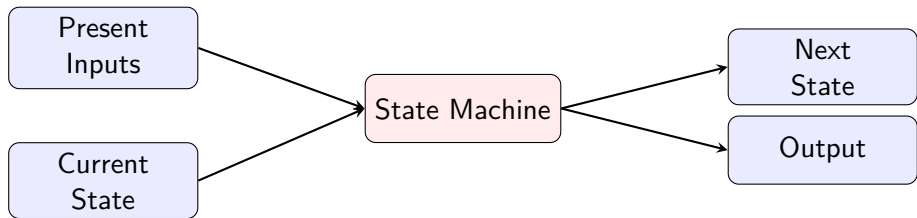
From Behavioral Description to Digital Circuit

CS214 – Computer Organization

From Sequential Circuits to State Machines

What We Already Know

In a sequential circuit, the behavior depends on the **present input** and the **current state**.



Key Idea

A state machine first describes **what the system should do**. Later, we decide how to implement the states and transitions using flip-flops and combinational logic.

What is a State?

Intuition

A **state** represents the information from the past that is relevant for deciding what should happen next.

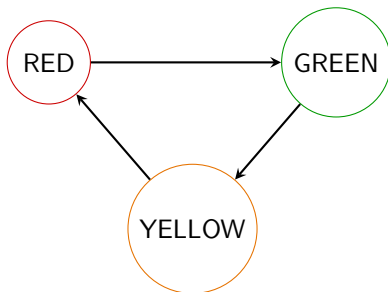
Past History $\longrightarrow$ Relevant Information $\longrightarrow$ State

Important

We do **not** need to remember the complete past. We only keep the information required to determine future behavior.

Simple Example: Traffic Light

Suppose three lamps glow cyclically with a fixed time interval.



RED → GREEN → YELLOW → RED

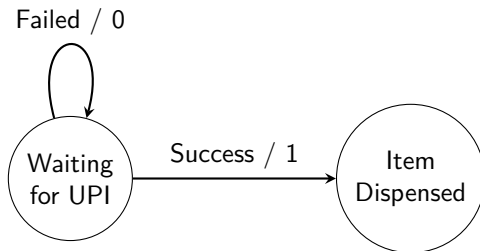
Observation

The glowing lamp is determined only by the **current state**.

Output depends only on state $\Rightarrow$ Moore Machine

Example: Hostel Vending Machine with UPI Payment

A student selects an item and makes a UPI payment.



Transition label: Input / Output

Failed/0

Success/1

Output depends on state + input $\Rightarrow$ Mealy Machine

Moore vs. Mealy Machine

Moore Machine

Output depends only on the current state:

$$Z = \lambda(S)$$

State $\longrightarrow$ Output

Mealy Machine

Output depends on the current state and input:

$$Z = \lambda(S, X)$$

State + Input $\longrightarrow$ Output

For both types, the next state is determined from

$$S^+ = \delta(S, X).$$

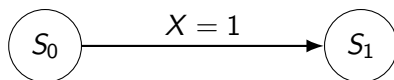
Deterministic vs. Non-Deterministic State Machine

Deterministic FSM

For every combination of

(current state, input),

there is **exactly one next state**.



Non-Deterministic FSM

The same state/input combination may allow more than one possible next state.

In synchronous digital circuit design, we normally synthesize **deterministic** FSMs.

How Do We Represent an FSM?

An FSM is commonly represented in two equivalent forms.

State Transition Diagram

- States are shown as circles.
- Directed arrows show transitions.
- Labels specify the input and, for a Mealy machine, the output.

State Table

For every present state and input, the table specifies

- the next state,
- the corresponding output.

State Diagram $\iff$ State Table

Example: Detect Three Consecutive 1's

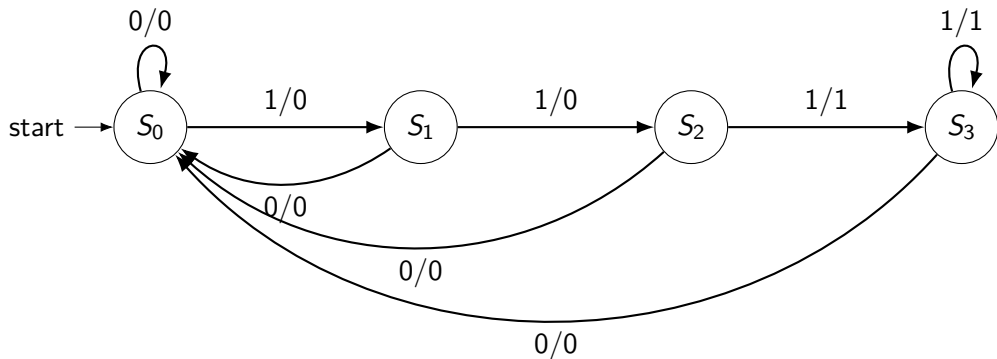
Problem

A serial bit stream X is applied one bit per clock cycle. Generate $Z = 1$ after detecting three or more consecutive 1's.

What information from the past do we need to remember?

State	Meaning
S_0	No useful consecutive 1 has been seen
S_1	One consecutive 1 has been seen
S_2	Two consecutive 1's have been seen
S_3	Three or more consecutive 1's have been seen

Three Consecutive 1's: State Diagram



Here each transition label is written as

input/output .

Three Consecutive 1's: State Table

Present State	$X = 0$	$X = 1$
	Next State, Z	Next State, Z
S_0	$S_0, 0$	$S_1, 0$
S_1	$S_0, 0$	$S_2, 0$
S_2	$S_0, 0$	$S_3, 1$
S_3	$S_0, 0$	$S_3, 1$

Same Behavior, Different Representation

The state diagram and state table describe exactly the same machine.

Formal Definition of a Deterministic FSM

A deterministic FSM can be written as

$$M = (\Sigma, \Gamma, S, s_0, \delta, \lambda)$$

where

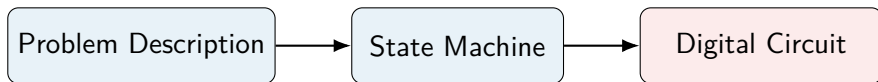
Σ	set of input combinations / input alphabet
Γ	set of output combinations / output alphabet
S	finite set of states
$s_0 \in S$	initial state
δ	state-transition function
λ	output function

$$\delta : S \times \Sigma \rightarrow S$$

For a Mealy machine, $\lambda : S \times \Sigma \rightarrow \Gamma$; for a Moore machine, $\lambda : S \rightarrow \Gamma$.

Main Question: Can We Build Such a Machine?

So far, the FSM describes **behavior**.

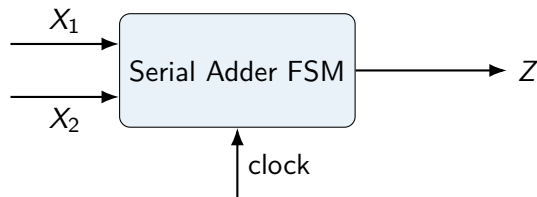


Question

Can we implement the state machine using the flip-flops and combinational circuits that we already know?

Example: Two-Input Serial Adder

Two binary numbers arrive serially, one pair of bits per clock cycle.



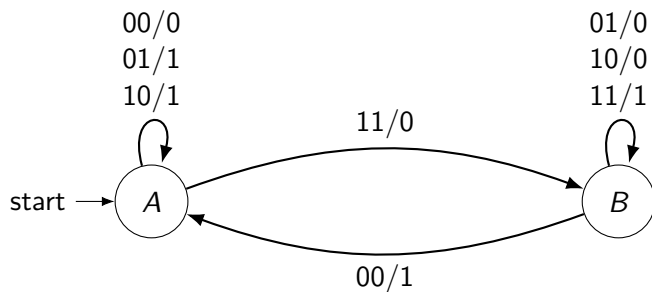
What Must Be Remembered?

The sum bit can be produced immediately from X_1 , X_2 , and the previous carry. Therefore, the machine only needs to remember the **carry**.

$$S_0 : C = 0, \quad S_1 : C = 1$$

Serial Adder: State Machine

Let A denote carry 0 and B denote carry 1.



Transition label:

$$X_1X_2/Z$$

Serial Adder: State Table

Present State	00	01	10	11
	Next State, Z			
A	$A, 0$	$A, 1$	$A, 1$	$B, 0$
B	$A, 1$	$B, 0$	$B, 0$	$B, 1$

Interpretation

For example, if the present carry is 1 and $X_1X_2 = 00$, then

$$0 + 0 + 1 = 1,$$

so $Z = 1$ and the next carry becomes 0.

Serial Adder: State Assignment

There are only two states, so one state bit is sufficient.

$$A \equiv Q = 0, \quad B \equiv Q = 1$$

Q X_1X_2	Q^+				Z			
	00	01	10	11	00	01	10	11
0	0	0	0	1	0	1	1	0
1	0	1	1	1	1	0	0	1

Key Point

State assignment converts symbolic states into binary values that can be stored in flip-flops.

Serial Adder: D Flip-Flop Input

For a D flip-flop,

$$D = Q^+$$

Q	X_1X_2	Q^+	D
0	00	0	0
0	01	0	0
0	10	0	0
0	11	1	1
1	00	0	0
1	01	1	1
1	10	1	1
1	11	1	1

$$D = X_1X_2 + Q(X_1 + X_2)$$

Serial Adder Using a D Flip-Flop

For a D flip-flop,

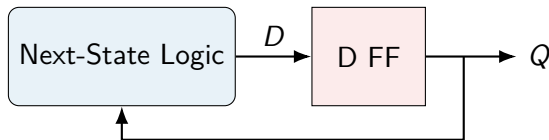
$$D = Q^+.$$

From the transition table,

$$D = X_1X_2 + Q(X_1 + X_2)$$

The output is the sum bit:

$$Z = X_1 \oplus X_2 \oplus Q$$



Serial Adder: JK Flip-Flop Inputs

Using the JK excitation requirements:

$$\begin{array}{ll} 0 \rightarrow 0 : J = 0 & 0 \rightarrow 1 : J = 1 \\ 1 \rightarrow 0 : K = 1 & 1 \rightarrow 1 : K = 0 \end{array}$$

Q	X_1X_2	Q^+	J	K
0	00	0	0	–
0	01	0	0	–
0	10	0	0	–
0	11	1	1	–
1	00	0	–	1
1	01	1	–	0
1	10	1	–	0
1	11	1	–	0

$$J = X_1X_2$$

$$K = \overline{X_1}\overline{X_2}$$

Same Serial Adder Using a JK Flip-Flop

The **FSM does not change**. Only the implementation of the state memory changes.

Using the JK excitation requirements,

$$J = X_1 X_2$$

$$K = \overline{X_1} \overline{X_2}$$

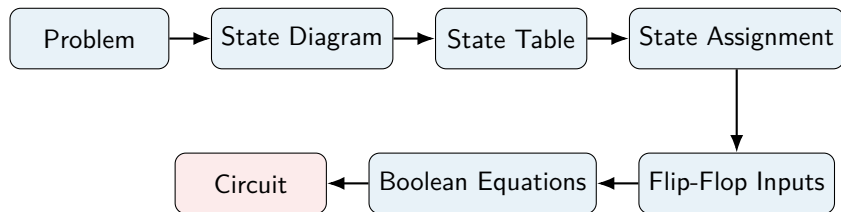
while the output remains

$$Z = X_1 \oplus X_2 \oplus Q.$$

Important

The state diagram and state table describe the **behavior**. D, JK, T, or SR flip-flops are different ways of **implementing** the same behavior.

General FSM Synthesis Flow



Synthesis

Synthesis means converting the behavioral description of an FSM into a realizable sequential circuit.

Second Example: Sequence Detector for 0110

Problem

A circuit accepts a serial bit stream X and produces $Z = 1$ whenever the pattern

0110

appears. At all other times, $Z = 0$.

Overlapping occurrences should also be detected.

X	0	1	1	0	1	1	0	0	1	1	0
Z	0	0	0	1	0	0	1	0	0	0	1

0110 Detector: How Do We Choose the States?

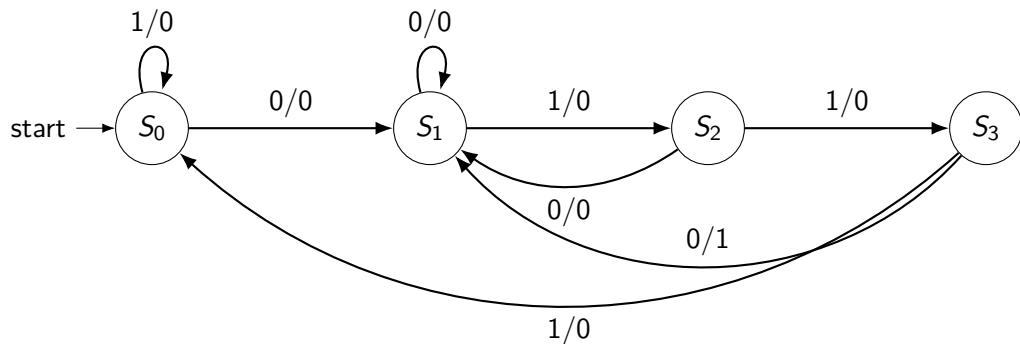
Each state remembers how much of the desired pattern has already been matched.

State	Meaning
S_0	No useful part of the pattern has been matched
S_1	The suffix 0 has been matched
S_2	The suffix 01 has been matched
S_3	The suffix 011 has been matched

State = Relevant Past

We do not store the complete input stream. We store only the part that can affect whether the next input completes the pattern.

0110 Detector: State Diagram



When S_3 receives 0, the sequence 0110 has just been completed, so $Z = 1$.

0110 Detector: State Table

Present State	$X = 0$	$X = 1$
	Next State, Z	Next State, Z
S_0	$S_1, 0$	$S_0, 0$
S_1	$S_1, 0$	$S_2, 0$
S_2	$S_1, 0$	$S_3, 0$
S_3	$S_1, 1$	$S_0, 0$

Overlap

After detecting 0110, the final 0 can also serve as the first 0 of a new occurrence. Therefore the machine moves to S_1 , not S_0 .

0110 Detector: State Assignment

Four states require two state bits.

$$S_0 = 00, \quad S_1 = 01, \quad S_2 = 10, \quad S_3 = 11$$

Present State $Q_1 Q_0$	Next State $Q_1^+ Q_0^+$		Z	
	$X = 0$	$X = 1$	$X = 0$	$X = 1$
00	01	00	0	0
01	01	10	0	0
10	01	11	0	0
11	01	00	1	0

Two Flip-Flops $\Rightarrow$ Two Next-State Functions

For the 0110 detector,

$$S_0 = 00, \quad S_1 = 01, \quad S_2 = 10, \quad S_3 = 11.$$

Two state bits $\Rightarrow$ two flip-flops: Q_1 and Q_0 .

PS		Input X	NS		D-FF Inputs	
Q_1	Q_0		Q_1^+	Q_0^+	D_1	D_0
0	0	0	0	1	0	1
0	0	1	0	0	0	0
0	1	0	0	1	0	1
0	1	1	1	0	1	0
1	0	0	0	1	0	1
1	0	1	1	1	1	1
1	1	0	0	1	0	1
1	1	1	0	0	0	0

For a D flip-flop, $D = Q^+$. Therefore,

0110 Detector Using D Flip-Flops

For D flip-flops,

$$D_1 = Q_1^+, \quad D_0 = Q_0^+.$$

From the state table,

$$D_1 = X(Q_1 \oplus Q_0)$$

$$D_0 = \overline{X} + Q_1 \overline{Q_0}$$

and the output is

$$Z = \overline{X} Q_1 Q_0.$$

Meaning

The combinational logic calculates the next state and output; the two D flip-flops store the current state.

0110 Detector Using JK Flip-Flops

The same state machine can also be implemented using two JK flip-flops.

Using the JK excitation table, one minimized set of excitation equations is

$$J_1 = XQ_0, \quad K_1 = \overline{X} + Q_0$$

$$J_0 = \overline{X} + Q_1, \quad K_0 = X$$

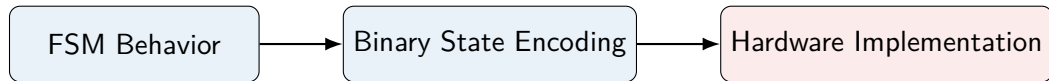
The output equation is

$$Z = \overline{X}Q_1Q_0$$

Same Behavior

Changing the flip-flop type changes the excitation equations, but it does not change the FSM specification.

Behavior vs. Implementation



Behavioral Level

- States
- Transitions
- Inputs
- Outputs

Hardware Level

- Flip-flops
- Excitation equations
- Combinational logic
- Clock

State machine describes behavior

State table converts behavior into equations

Flip-flops + combinational logic implement the machine

The Complete Story

Problem $\rightarrow$ FSM $\rightarrow$ State Assignment $\rightarrow$ Boolean Logic $\rightarrow$ Sequential Circuit

- M. Morris Mano and Michael D. Ciletti, *Digital Design*.
- CS214 course material: previous weeks on combinational and sequential circuits, latches, and flip-flops.